# Upwork Interview Prep — Multi-Agent AI Chatbot Developer Position

> **Bu doküman sadece interview hazırlığı içindir. .gitignore'a eklendi, GitHub'da görünmez.**

# 1. Cover Letter / Initial Application Message

Hi,

I'd like to apply for the multi-agent AI chatbot developer position. I'm a Python developer with hands-on experience building production-grade multi-agent AI systems using FastAPI, CrewAI, and LLM APIs — deployed with Docker Compose on PostgreSQL.

My most relevant project is Confluence Trade Crew — a multi-agent decision-support platform I built from scratch. It features a 6-agent CrewAI pipeline orchestrated via FastAPI, real-time agent telemetry over Redis Pub/Sub → WebSockets, multi-provider LLM routing (Anthropic, OpenAI, Gemini, GitHub Models, Ollama), and a vectorized backtesting engine — all containerized with Docker Compose (5 services: Python AI service, .NET API, React frontend, PostgreSQL, Redis).

Here's what I bring that directly maps to your requirements:

- Python & FastAPI: My entire AI service layer is built on FastAPI with Pydantic schemas, async endpoints, and strategy-pattern based LLM routing.
- Multi-agent AI workflows: I designed and implemented a 6-agent pipeline where agents run in parallel with explicit dependency management (Data Agent → TA + News + OnChain [parallel] → Risk → Orchestrator). I've solved real-world agentic issues like ReAct loop failures, LLM hallucination hardening, rate limit distribution, and subprocess state management.
- LLM APIs & routing: Built a Factory Pattern LLM router that supports 5 providers with zero-code switching via environment variables. Each agent can use a different provider/model.
- Docker & Linux: The entire system runs on Docker Compose with multi-stage builds, health checks, and internal service networking.
- PostgreSQL: Used as the primary data store (managed via .NET EF Core, but I understand the schema design, migrations, and query patterns thoroughly).
- Debugging & problem-solving: I've documented 25+ production bugs I solved — from MCP subprocess memory leaks to CrewAI telemetry event parsing to LLM provider async/sync conflicts.

GitHub: https://github.com/aeren23/confluence-trade-crew

I can commit 8 hours per day and am available to start immediately.

```
Looking forward to discussing how I can help ship your Telegram bot.

Best regards,
[Your Name]
```

# 2. Skill-to-Job Requirement Mapping

Bu tablo görüşmede her soruya "benim projemde bunu yaptım" diye somut cevap vermeni sağlar:

| Job Requirement | Your Direct Experience (Confluence Trade Crew) | Code Reference |
|---|---|---|
| **Strong Python skills** | Entire AI service (~4,000 LOC Python): FastAPI endpoints, CrewAI agent definitions, MCP server protocol handler, pandas vectorized backtesting, Pydantic schemas, async/sync Redis pub/sub | `ai-service/app/` |
| **FastAPI** | 3 FastAPI routers ( `/analyze` , `/backtest` , `/review_trade` ), CORS middleware, health checks, async request handlers, Pydantic request/response models | `ai-service/app/main.py` , `ai-service/app/api/` |
| **LLMs & AI agents** | 6 CrewAI v2 agents with per-agent prompt engineering, ReAct tool exploration, LLM Factory Pattern supporting 5 providers | `ai-service/app/crew/` , `ai-service/app/llm/factory.py` |
| **Multi-agent chatbot logic** | Explicit dependency graph: Data → TA+News+OnChain (parallel | `ai-service/app/crew/confluence_crew.py` |

| Job Requirement | Your Direct Experience (Confluence Trade Crew) | Code Reference |
| --- | --- | --- |
| | `async_execution=True` ) → Risk → Orchestrator. Step callbacks for live agent reasoning. Task-level `context` parameter for data flow | |
| **Read existing codebase** | Joined the project at various phases, integrated 6 phases of accumulated code, resolved architectural conflicts between CrewAI internals and FastAPI's async model | `docs/state.md` — 6 phases documented |
| **Docker & Linux** | Docker Compose orchestrating 5 services (db, redis, ai-service, api, frontend), multi-stage Dockerfiles, health checks, SOCKS5 proxy routing for containers | `docker-compose.yml` , `ai-service/Dockerfile` |
| **PostgreSQL** | Schema design: 7 entities (Trade, Analysis, Pair, UserSettings, StrategyTemplate, TradeReview, AnalysisAccuracy), migrations, GIN indexes on JSONB columns | `api/src/Confluence.Domain/Entities/` |
| **Debug & deliver** | 25+ documented bugs solved across 6 weeks — MCP cache survival, CrewAI telemetry parsing, direction- | `docs/state.md` § Solved Problems |

| Job Requirement | Your Direct Experience (Confluence Trade Crew) | Code Reference |
|---|---|---|
| | aware SL/TP, rate limit distribution, SSL/DNS bypass | |
| **English communication** | All code, comments, docstrings, and documentation written in English | Entire codebase |
| **TypeScript/React (bonus)** | React 18 frontend: 18 pages, lightweight-charts integration, Zustand state, SignalR WebSocket client, CSS Modules | `frontend/src/` |

# 3. Key Technical Talking Points

## 3.1 "Tell me about your multi-agent experience"

**Cevap şablonu:**

> "I built a 6-agent AI pipeline from scratch using CrewAI v2 on top of FastAPI.
> The agents are specialized — Data Agent fetches market data, Technical Analysis Agent computes indicators, News Agent gathers sentiment, OnChain Agent checks derivatives data, Risk Agent calculates position sizing, and the Orchestrator synthesizes everything into a structured JSON output.
>
> The key challenge was managing **agent dependencies**. TA, News, and OnChain agents are independent of each other, so they run in parallel using CrewAI's `async_execution=True`. But the Risk Agent needs all three outputs before it can calculate — so it's the **join point** in the task graph.
>
> I also learned that CrewAI's `output_pydantic` constraint kills the ReAct reasoning loop — agents stop exploring tools and just try to produce the schema. I solved this by letting sub-agents output plain text and only enforcing JSON on the final Orchestrator."

**Teknik detay (sorulursa):**

- `confluence_crew.py` → `@task` decorators with `context=[]` for dependency injection
- `async_execution=True` on TA, News, OnChain tasks
- `step_callback` parsing AgentAction/AgentFinish objects for live telemetry

## 3.2 "How do you handle LLM routing and provider management?"

> "I built a Factory Pattern where each agent can use a different LLM provider. The `LLMFactory` reads a model string like `anthropic/claude-3-5-haiku-20241022` or `github/gpt-4.1-nano`, extracts the provider prefix, resolves the correct API key from environment variables, and returns a configured CrewAI LLM instance.
>
> This was critical because GitHub Models has a 150 requests/day per-model limit. By distributing one model per agent across 5 different GitHub models, I effectively got 750 requests/day without any code changes — just `.env` updates."

**Teknik detay:** `_PROVIDER_KEY_MAP` dict → `_resolve_api_key()` → `LLM(**kwargs)` return

## 3.3 "How do you handle conversation management / state?"

> "In my system, the AI service is **completely stateless** — it takes a request, runs the pipeline, returns JSON, and forgets everything. All persistence is handled by the .NET API layer writing to PostgreSQL.
>
> But within a single pipeline execution, agents need to share data. For example, the Data Agent fetches OHLCV candles and other agents need to compute indicators on the same data. CrewAI spawns MCP tool calls as separate subprocesses, destroying in-memory state each time.
>
> I solved this with a **Parquet file-based cache** — the Data Agent writes the DataFrame to a Parquet file keyed by a session UUID. Other agents' tool calls read from this file. The cache is cleared after each analysis session in a `try/finally` block."

**Telegram bot ile paraleli:** Telegram bot'unda da conversation state management olacak — session bazlı state tracking, Redis'te conversation context tutma gibi konular bu deneyimle doğrudan örtüşüyor.

## 3.4 "Can you debug existing codebases?"

> "Absolutely. I've documented over 25 production bugs I debugged in this project.
> Let me give you three concrete examples:
>
> 1. **CrewAI telemetry parsing:** The live agent thought stream was showing 'Processing...' instead of actual thoughts. Root cause: CrewAI's step_callback passes AgentAction objects where the thought text is buried in the `.log` attribute before the `Action:` line — not in a `.thought` attribute like the docs suggest. I rewrote the callback parser to extract thoughts from `.log`, tool calls from `.tool` / `.tool_input`, and final outputs from `.return_values`.
> 2. **Async/sync conflict:** CrewAI step_callbacks run synchronously in a thread pool, but my Redis telemetry publisher was async. Calling `asyncio.get_running_loop()` in a thread raises RuntimeError. I added a separate synchronous Redis client with thread-safe lazy initialization for step_callbacks, keeping the async client for FastAPI handlers.
> 3. **LLM hallucination hardening:** The Risk Agent was calculating long-position stop-loss for bearish analysis. I traced the issue to the prompt only describing long calculations. I rewrote the prompt to read the TA sentiment_score first and branch to LONG/SHORT/NEUTRAL
> calculation paths."

## 3.5 "What's your Docker experience?"

> "The entire system runs on Docker Compose with 5 services: PostgreSQL, Redis, Python AI Service,
> .NET API, and React frontend. Each service has its own Dockerfile — the Python service uses `python:3.12-slim` with a healthcheck, the .NET API uses multi-stage builds, and the frontend uses Node 20 for the Vite build.
>
> I also solved a real-world networking issue: Docker containers running in WSL2 don't share the host's Cloudflare WARP VPN tunnel. I configured SOCKS5 proxy routing through `host.docker.internal:40000` so containers can reach external APIs through the host's VPN."

## 3.6 "Tell me about your FastAPI experience"

> "My AI service is a FastAPI application with three main routers:

> - `POST /analyze` — triggers the multi-agent pipeline, returns structured JSON
> - `POST /backtest` — runs vectorized historical simulation using pandas
> - `POST /review_trade` — direct LLM call for trade evaluation
>
> I use Pydantic Settings for configuration management — all API keys and model selections are loaded from environment variables with proper defaults. The analyze endpoint uses Pydantic request/response models with full type validation.
>
> For the telemetry system, I needed both sync and async paths because CrewAI callbacks are synchronous but FastAPI is async. I built a dual-client Redis publisher that handles both contexts safely."

# 4. Telegram Bot Adaptability — Nasıl Bağlayacaksın

İş ilanında Telegram bot var, sende yok. Ama aşağıdaki bağlantıları kurabilirsin:

| Their Need | Your Transferable Experience |
|---|---|
| Telegram Bot API integration | "I haven't used python-telegram-bot specifically, but I've built the same patterns — async event handlers, user session management, message routing. The Telegram Bot API is well-documented and I can pick it up in a day." |
| Conversation management | "My pipeline manages multi-step agent conversations with explicit dependency ordering. The concept is the same — track conversation state, route messages to the right handler, maintain context across turns." |
| Agent logic & workflows | "This is exactly what I built. My 6-agent pipeline with parallel execution, dependency injection, and step-level telemetry is a production multi-agent workflow." |
| LLM routing | "My LLMFactory supports 5 providers with zero-code switching. If their bot needs to route between different models for different conversation types, I've already solved this." |
| Preparing for public launch | "I've built the entire DevOps stack — Docker Compose, health checks, graceful error handling, fallback mechanisms, environment-based configuration. I know what production readiness looks like." |

| Their Need | Your Transferable Experience |
|---|---|
| Supabase/PostgreSQL | "My system uses PostgreSQL with 7 entities, complex relationships, and migrations. Supabase is PostgreSQL with extras — the query patterns and schema design transfer directly." |

**Telegram Bot bilgi eksikliği konusunda:**

> "I haven't used python-telegram-bot or aiogram in a production project yet, but my multi-agent FastAPI architecture is architecturally identical to what a Telegram bot needs — async message handlers, user session state, conversation routing, and LLM integration. I can be productive with the Telegram Bot API within a day of onboarding."

# 5. Questions to Ask Them

Görüşmede soru sormak çok önemli — ilgi ve profesyonellik gösterir:

1. **"What LLM provider(s) is the bot currently using, and is there routing between models?"**
   → Bu soruyla LLMFactory deneyimini doğal olarak gösterirsin.
2. **"How are agent conversations structured — is it a single-turn or multi-turn pipeline?"**
   → Multi-agent pipeline deneyimini bağlarsın.
3. **"What's the current bottleneck — is it more about agent logic, infrastructure, or UX?"**
   → Problem-solving yaklaşımını gösterirsin.
4. **"Is the bot deployed with Docker, and what does the CI/CD pipeline look like?"**
   → Docker bilgini doğal şekilde sergilersin.
5. **"What does 'public launch' mean for you — is there a beta phase, or are we going straight to production?"**
   → Profesyonel bir soru — production readiness deneyimini gösterir.
6. **"How is conversation state currently managed — in-memory, Redis, database?"**
   → Redis Pub/Sub ve state management deneyimini bağlarsın.

# 6. Rate Negotiation Strategy

İlan $12-20/hr diyor. Stratejin:

1. **$18-20/hr ile başla.** Projen mid-level'ı aşan teknik derinlik gösteriyor.

2. **Gerekçelen:**
    - "I have a production multi-agent system with 6 agents, not just tutorial-level LLM wrappers"
    - "I've solved 25+ real-world agentic system bugs — I won't need hand-holding"
    - "I bring FastAPI + Docker + PostgreSQL as a complete stack, not just Python"
3. **Esneklik göster:** "I'm open to starting at $18 and adjusting based on the value I deliver in the first two weeks."
4. **Uzun vadeli kancayı at:** "I'm very interested in the long-term opportunity mentioned in the listing. I'm looking for a team to grow with, not just a one-off gig."

# 7. GitHub Profile Checklist

Görüşmeden önce GitHub profilinin şunlara sahip olduğundan emin ol:

- [x] `confluence-trade-crew` repo'su public mi? → Emin ol
- [ ] GitHub profil README'si var mı? → Yoksa kısa bir tane ekle
- [ ] Pinned repositories arasında `confluence-trade-crew` var mı?
- [ ] Repo'da yeni README (388 satır, 3 görsel, badges) canlı mı?
- [ ] LICENSE dosyası (MIT) repo'da var mı? → Yoksa ekle

# 8. Red Flags to Watch For

- Eğer "free trial task" istiyorlarsa → Makul bir süre sınırı koy (2-4 saat max)
- Eğer codebase'lerini paylaşmak istemiyorlarsa görüşmeden önce → Dikkatli ol
- Eğer $12/hr altına düşürmeye çalışırlarsa → Nazikçe reddet, projeni referans göster
- Eğer "full-time exclusive" istiyorlarsa → Net bir şekilde saatlik bazda çalıştığını belirt

# 9. Quick Reference: Project Stats

Görüşmede kullanabileceğin somut rakamlar:

| Metric | Value |
|---|---|
| Total Python LOC (ai-service) | ~4,000+ |
| CrewAI Agents | 6 (Data, TA, News, OnChain, Risk, Orchestrator) |
| MCP Tools | 12+ (data, indicator, news, on-chain) |
| FastAPI Endpoints | 3 routers, 6+ endpoints |
| Docker Services | 5 (db, redis, ai-service, api, frontend) |
| LLM Providers Supported | 5 (Anthropic, OpenAI, Gemini, GitHub Models, Ollama) |
| Agent Prompt Files | 6 individual prompt engineering files |
| Documented Bugs Solved | 25+ |
| Development Duration | ~6 weeks (13 June → 29 June 2026) |
| PostgreSQL Entities | 7 (Trade, Analysis, Pair, UserSettings, Strategy, Review, Accuracy) |
| Frontend Pages | 18 (Dashboard, Trading, Portfolio, Backtest, Compare, etc.) |
| React Components | 20+ |
| Project Documentation | 5 dedicated docs (architecture, agents, MCP tools, DB schema, setup) |

# 10. One-Liner Pitch

Eğer kendini tek cümleyle tanıtman gerekirse:

> "I'm a Python developer who built a production 6-agent AI pipeline with FastAPI, CrewAI, Redis Pub/Sub, and Docker Compose — I understand multi-agent orchestration, LLM routing, and agentic debugging at a level that goes well beyond tutorials."